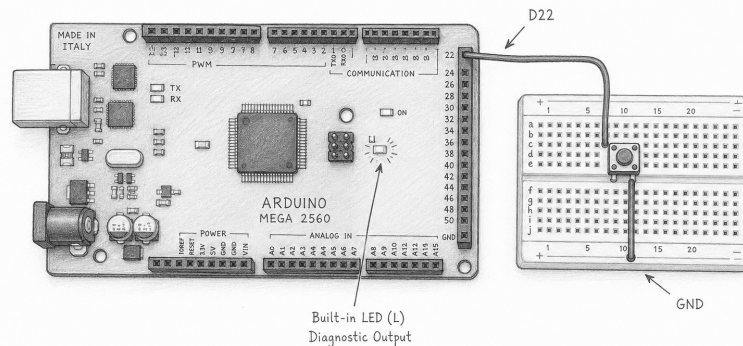


Digital Input

A safe pull-up switch on Arduino Mega 2560

Observe the pin first; interpret human intent second.

Lesson	002	Time	70–95 minutes
Requires	Lesson 001: Digital Output	Board	Arduino Mega 2560
Outcome	Read and diagnose one grounded switch	Evidence	Truth table, serial trace, host test, three trials



Orientation only. The exact schematic and connection table below govern wiring; verify D22 and GND before applying power.

1 Why input comes this early

An output lets firmware report what it thinks. An input lets a person change what firmware does. Together they form the smallest useful debugging panel. This lesson keeps the two ideas separate: the new component reports the *electrical* pin level without debouncing or inventing button events. Lesson 003 will build those policies on top.

Learning targets

By the end, you can:

- wire a normally-open switch from a Mega input to ground;
- explain why an internal pull-up gives an unpressed pin a known level;
- distinguish a sampled electrical level from an application's meaning;
- diagnose open, stuck-low, and noisy input evidence;
- drive the same input sequence through a deterministic host fake;
- explain how ownership and RAI leave the pin passive at shutdown.

2 Safety and equipment

Safety checkpoint. Disconnect USB, barrel power, batteries, programmers, and powered modules before changing wiring. The switch must connect D22 to GND—never to an output. Do not connect 5 V directly to GND. Use only a dry-contact switch in this lesson. Do not use mains-powered, automotive, pyrotechnic, or unknown equipment.

Required: Arduino Mega 2560, USB data cable, breadboard, normally-open momentary pushbutton, two jumper wires, and the Lesson 001 diagnostic LED. A multimeter is useful but optional.

Before power: ☐ switch straddles the breadboard center gap; ☐ one selected switch terminal reaches D22; ☐ its opposite terminal reaches Mega GND; ☐ there is no 5 V jumper; ☐ continuity is open when released and closed when pressed.

3 Predict the electrical behavior

The Mega's internal pull-up is a weak resistor from the input to the board supply. It establishes HIGH when the switch is open. Pressing the switch provides a much lower-resistance path to ground, so the pin reads LOW. No external resistor is required for this exercise.

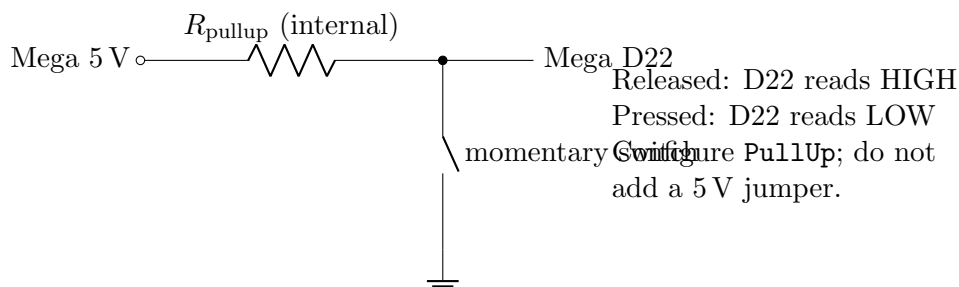
Switch	D22 path	Raw level	Human interpretation
Released	through pull-up to supply	HIGH	switch open
Pressed	through switch to GND	LOW	switch closed

Prediction: If the ground wire falls out, the input will probably remain _____. Why?

Prediction: If D22 is shorted to ground, what will the raw trace show?

Vocabulary boundary. `DigitalInput` reports `Level::High` or `Level::Low`. *Pressed*, *released*, *pressed edge*, and *debounced* belong to a `Button` component. The application may compare a level with `LOW`, but `DigitalInput` does not attach human meaning to it.

4 Build from the exact schematic



From	To	Check with power disconnected
Mega D22	one side of switch	continuous to the chosen leg
opposite switch side	Mega GND	continuous to a GND pin
D22	GND	open released; near zero ohms pressed

Four-leg tactile switches connect the two legs on each side internally. Rotating one by 90° can make the input permanently grounded. Use continuity mode or the switch datasheet; do not trust physical symmetry.

5 Read state deliberately

The canonical `Lesson002DigitalInput.ino` composes the input with Lesson 001's output:

```
constexpr adk::PinId ButtonPin = 22;

adk::Runtime      runtime;
adk::DigitalInput buttonInput (runtime.resources (), ButtonPin, adk::Pull::Up);
adk::MonoLed      led          (runtime.resources (), LED_BUILTIN);

void setup ()
{
    buttonInput.initialize ();
    led.initialize          ();
}

void loop ()
{
    buttonInput.update ();

    const bool pressed = buttonInput.read () == adk::Level::Low;
    led.set (pressed);
}
```

The variable is named `button` because of its circuit role; its type remains `DigitalInput`. `update()` samples hardware and stores the level. `read()` returns that stored level without a surprise hardware access. `sample()` performs an immediate hardware read without changing the stored level. Prefer `update()` plus `read()` for coherent application frames; reserve `sample()` for deliberate diagnostics.

The compact example omits error reporting to expose the central composition. In an application, compare every initialization result explicitly:

```
const adk::Status buttonStatus = button.initialize ();
const adk::Status ledStatus    = led.initialize    ();

if (!buttonStatus.ok () || !ledStatus.ok ())
{
    button.shutdown ();
    led.shutdown    ();
    return;
}
```

RAII contract: construction stores configuration without touching hardware; successful `initialize()` claims and configures D22; repeated `shutdown()` is harmless; destruction releases the claim and restores a passive input state. Internal code reports failure with `Status`, not exceptions, while cleanup remains correct if surrounding application code throws.

6 Observe before interpreting

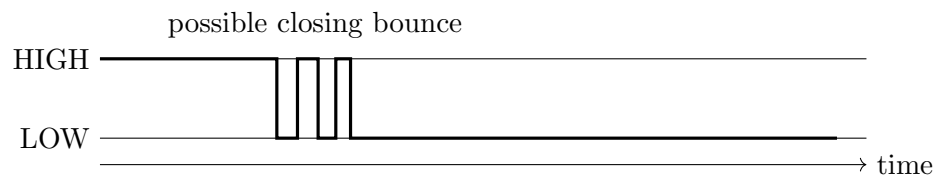
Upload with the switch released. Open the serial monitor at 115200 baud. Collect at least ten samples for each state rather than reporting one favorable line.

Trial	Switch	Expected raw	Observed raw	Diagnostic LED / notes
1	released	HIGH	_____	_____
1	pressed	LOW	_____	_____
2	released	HIGH	_____	_____
2	pressed	LOW	_____	_____
3	released	HIGH	_____	_____
3	pressed	LOW	_____	_____

Diagnostic mapping: the canonical sketch turns the Lesson 001 LED on when `button.read () == adk::Level::Low`. The LED is a view of the stored sample, not a second input read. If a serial view says LOW and the LED is off, investigate the output path; if both are wrong, investigate input wiring.

7 Contact bounce is evidence, not yet a bug

A mechanical contact may alternate HIGH–LOW several times while closing. At a 100 ms reporting interval you may miss this; at a faster diagnostic interval you may see it. `DigitalInput` must preserve raw evidence. It must not silently delay, filter, or synthesize events.



Lesson 003 will consume raw samples at explicit times, require stability for a configured interval, and expose button events. Keeping that policy in a separate component allows other users—limit switches, jumpers, and digital sensors—to observe unfiltered levels.

8 Deterministic host test

The API and lifecycle below are **host-verified**. The physical circuit remains **experimental** until you complete and record the Mega acceptance checks. Host tests prove component rules without USB, wall-clock delays, or nondeterministic contact timing.

```
fake::reset ();
adk::ResourceRegistry resources;
fake::setDigitalInput (22, HIGH);
adk::DigitalInput input (resources, 22, adk::Pull::Up);

require (input.initialize ().ok ());
```

```

require (input.read () == adk::Level::High);

fake::setDigitalInput (22, LOW);
require (input.sample () == adk::Level::Low);
require (input.read () == adk::Level::High);

input.update ();
require (input.read () == adk::Level::Low);

input.shutdown ();
input.shutdown ();

```

This mirrors the repository's fake-Arduino tests: `sample()` observes LOW without mutating the cached HIGH; `update()` then stores LOW. Separate tests verify the exact `pinMode`, `digitalRead`, `claim-conflict`, `invalid-pin`, `repeated-initialization`, `repeated-shutdown`, and `destructor` traces. A conflict is reported as a status:

```

adk::DigitalInput owner (resources, 22, adk::Pull::Up);
adk::DigitalInput blocked (resources, 22, adk::Pull::Up);

require (owner.initialize ().ok ());
require (blocked.initialize ().error () == adk::StatusCode::ResourceBusy);

```

Preserve these assertions when extending the tests:

1. construction alone produces no hardware call;
2. initialization claims D22, selects INPUT_PULLUP, and samples;
3. repeated successful initialization produces no hardware call;
4. `sample()` reads hardware without changing `read()`;
5. `update()` reads hardware and changes `read()`;
6. shutdown is idempotent and restores INPUT;
7. destruction releases the claim;
8. a second owner reports `error() == StatusCode::ResourceBusy`;
9. an invalid pin reports `error() == StatusCode::InvalidPin` without touching hardware.

Extend the test: script HIGH, LOW, HIGH, LOW without elapsed time. Record the four results. Why must `DigitalInput` preserve all four?

9 Fault laboratory

Change one condition at a time, with power removed before each wiring change. Restore and recheck the exact schematic after every trial.

Safe fault		Expected evidence	What the evidence distinguishes
Disconnect switch-to-GND wire		always <code>Level::High</code>	open return path versus a working press
Move input configuration from pull-up to floating input		unstable or environment-dependent level	defined bias versus a floating conductor; restore immediately
Hold switch pressed during reset		<code>Level::Low</code> after initialization	startup state is sampled, not assumed
Temporarily slow reporting to 500 ms		fewer observations, same mapping	sampling/display rate versus electrical meaning

Safety checkpoint. Do not create faults by shorting an output, supply, or external signal. The only wiring fault used here is an open connection. A floating-input demonstration is brief, uses only D22 and the switch, and is restored to pull-up configuration before continuing.

10 Mega 2560 acceptance checklist

Board model: _____ port: _____

Toolchain version: _____

- ☐ Clean build succeeds with warnings treated according to project policy.
- ☐ Sketch size is recorded: flash _____ bytes; SRAM _____ bytes.
- ☐ Initialization succeeds with only one owner of D22.
- ☐ Released reports `Level::High` for ten samples.
- ☐ Pressed reports `Level::Low` for ten samples.
- ☐ Diagnostic LED agrees with the same stored sample.
- ☐ Three press/release trials match the prediction table.
- ☐ Ground-wire-open fault is detected from evidence and restored.
- ☐ Reset while pressed reports the actual pressed state.
- ☐ USB is disconnected before the circuit is dismantled.

11 Explain the result

Claim: In one sentence, state whether the pull-up input distinguishes the two switch states.

Evidence: Cite at least three observations, including raw level and the diagnostic output.

Reasoning: Use the pull-up and ground paths to explain why LOW means the switch is pressed in this circuit.

Limit: Why do these trials not prove that every brief raw transition is a deliberate human press?

12 Further work and references

Try one extension at a time:

- record a high-rate raw trace and mark possible bounce;
- use a suitable external pull-down and redraw the exact circuit before building;
- test claim conflicts by attempting two `DigitalInput` owners for D22;
- replace the switch with a documented, 5 V-compatible digital sensor;
- pass raw snapshots to the Button lesson without changing `DigitalInput`.

Use the HTML lesson for current repository commands, source links, API signatures, and errata. This printable field guide emphasizes wiring, predictions, evidence, and acceptance. Useful primary references:

- Arduino Mega 2560 Rev3 documentation and pinout: Arduino Mega 2560
- Arduino digital pin and pull-up tutorial: Digital Input Pull-Up
- ATmega2560 datasheet, I/O ports and pull-up behavior: Microchip ATmega2560
- ADK repository documentation: <https://spincyc.github.io/adk/>

Next: Lesson 003 composes `DigitalInput` with explicit time to create a debounced Button. After that lesson, the first interval project combines digital output, digital input, and button events in a deterministic reaction timer.